

МИНОБРНАУКИ РОССИИ
Федеральное государственное автономное
образовательное учреждение высшего образования
«Пермский государственный национальный
исследовательский университет»

Институт компьютерных наук и
технологий

ОТЧЁТ
по индивидуальной работе №2
по дисциплине «Языки программирования»
Вариант 1

Работу выполнил
студент группы ИТ-7,8-2025 1 курса
Антипин Дмитрий Константинович
«19» июня 2026 г.

Работу проверила
Шилина Алла Владимировна
«20» июня 2026 г.

Пермь 2026

Содержание

Постановка задачи.....	4
Алгоритм решения	4
Задача 1. Вычисление выражения	4
Задача 2. Почтальон Тритон	5
Структуры данных.....	6
Стек	6
Дерево (список смежности)	6
Архитектура программы.....	7
Пользовательский интерфейс	7
Главное меню	7
Подменю задачи 1	8
Подменю задачи 2	8
Тестирование	8
Блок 1. Задача 1 — базовые сценарии	8
Тест 1.1. Пример из задания	8
Тест 1.2. Простой минимум.....	9
Тест 1.3. Простой максимум	9
Тест 1.4. Две вложенные операции.....	9
Блок 2. Задача 1 — граничные случаи	9
Тест 2.1. Одинаковые аргументы	9
Блок 3. Задача 1 — обработка некорректных данных	10
Тест 3.1. Недопустимый символ.....	10
Тест 3.2. Несбалансированные скобки	10
Тест 3.3. Посторонние символы	10
Тест 3.4. Пустая строка	11
Блок 4. Задача 2 — базовые сценарии	11

Тест 4.1. Пример из задания	11
Тест 4.2. Все без писем.....	12
Тест 4.3. Одно гнездо	12
Блок 5. Задача 2 — граничные случаи	13
Тест 5.1. Корень без писем, дети с письмами	13
Тест 5.2. Цепочка, письма в конце	14
Тест 5.3. Три ветки от корня.....	15
Блок 6. Задача 2 — ошибки файлов	15
Тест 6.1. Файл не найден	15
Тест 6.2. Пустой файл.....	16
Тест 6.3. $M = 0$	16
Тест 6.4. Несоответствие строк	17
Блок 7. Интерфейс	17
Тест 7.1. Неверный пункт меню.....	17
Тест 7.2. Возврат из подменю	18
Тест 7.3. Выход из программы	18
Анализ результатов	18
Код программы	19

Постановка задачи

Целью работы является разработка программы на языке Python, реализующей вычисление математических выражений с операциями минимума и максимума, а также поиск оптимального маршрута почтальона-тритона. Программа использует собственные динамические структуры данных (стек и дерево), имеет модульную архитектуру, дружелюбный интерфейс с циклическим меню и обработку ошибок.

Задача 1. Пользователь вводит строку без пробелов, содержащую вложенные операции:

- $m(a,b)$ - минимум из a и b
- $M(a,b)$ - максимум из a и b

Аргументы — только целые положительные числа. Операции могут быть вложенными. Программа должна вычислить и вывести результат, используя структуру «стек». Строка считается корректной, пробелы не допускаются.

Пример: $M(15, m(16, 8)) \rightarrow 15$.

Задача 2. Программа считывает из файла описание M гнезд на дубе ($0 < M < 1000$).

Гнездо 1 расположено на первой развилке ствола. Для каждого гнезда известны: количество соседей, количество писем и номера соседних гнезд. Почтальон-тритон ползет по веткам, доставляя письма. Если его путь проходит через гнездо, в которое он не доставляет письма (писем нет или уже доставлены), он извиняется. Программа должна определить минимальное количество извинений, необходимое для доставки всех писем.

Алгоритм решения

Задача 1. Вычисление выражения

Разработан однопроходный алгоритм разбора строки слева направо с использованием стека:

1. Если текущий символ - m или M (операция), она помещается в стек.
2. Если текущий символ - цифра, извлекается все число целиком и помещается в стек.
3. Если текущий символ - $)$ (закрывающая скобка), из стека извлекаются два операнда и одна операция. Выполняется операция $\min$ или $\max$, результат помещается обратно в стек.
4. Остальные символы (скобка $($, запятая) пропускаются.
5. После обработки всей строки в стеке остается единственное число — ответ.

Псевдокод:

ФУНКЦИЯ `evaluate(expr)`:

СОЗДАТЬ `stack`

`i = 0`

ПОКА `i < ДЛИНА(expr)`:

`c = expr[i]`

```

ЕСЛИ c == 'm' ИЛИ c == 'M':
    stack.push(c); i++
ИНАЧЕ ЕСЛИ c — цифра:
    num = СЧИТАТЬ_ЧИСЛО(expr, i)
    stack.push(num)
ИНАЧЕ ЕСЛИ c == ')':
    b = stack.pop()
    a = stack.pop()
    op = stack.pop()
    ЕСЛИ op == 'm':
        stack.push(min(a, b))
    ИНАЧЕ:
        stack.push(max(a, b))
    i++
ИНАЧЕ:
    i++
ВЕРНУТЬ stack.pop()

```

Задача 2. Почтальон Тритон

Разработан рекурсивный алгоритм обхода дерева в глубину (DFS) от корня (узел 1):

1. Для каждого узла рекурсивно обходятся все соседи, кроме родительского.
2. Подсчитывается количество дочерних поддеревьев, содержащих хотя бы одно письмо (`children_with_mail`).
3. Если текущий узел имеет письма:
 - Если `children_with_mail > 1`, добавляется `children_with_mail - 1` извинений (каждый лишний заход в узел после доставки).
4. Если текущий узел без писем:
 - Если `children_with_mail > 0`, добавляется `children_with_mail` извинений (каждый проход через узел без доставки).
5. Функция возвращает `True`, если в поддереве есть письма, иначе `False`.
6. Сумма по всем узлам дает минимальное количество извинений.

Псевдокод:

ФУНКЦИЯ `min_apologies()`:

```

visited = ПУСТОЕ_МНОЖЕСТВО
apologies = 0

```

ФУНКЦИЯ `dfs(v, parent)`:

```

visited.ДОБАВИТЬ(v)
children_with_mail = 0
ДЛЯ КАЖДОГО to ИЗ neighbors[v]:

```

```

ЕСЛИ to != parent:
    ЕСЛИ dfs(to, v):
        children_with_mail++
ЕСЛИ letters[v] > 0:
    ЕСЛИ children_with_mail > 1:
        apologies += children_with_mail - 1
    ВЕРНУТЬ True
ИНАЧЕ:
    ЕСЛИ children_with_mail > 0:
        apologies += children_with_mail
    ВЕРНУТЬ True
    ВЕРНУТЬ False

dfs(1, -1)
ВЕРНУТЬ apologies

```

Структуры данных

Стек

Стек реализован в виде класса Stack на основе встроенного списка Python. Это динамическая структура данных, работающая по принципу LIFO (Last In, First Out — последний пришел, первый ушел).

Основные методы:

- `push(item)` - добавляет элемент на вершину стека
- `pop()` - удаляет и возвращает элемент с вершины стека
- `is_empty()` - проверяет, пуст ли стек

Использование собственного класса вместо встроенного списка напрямую соответствует требованию задачи о работе с динамическими структурами данных через собственные классы. Инкапсуляция защищает внутреннее представление от прямого доступа и позволяет при необходимости изменить реализацию (например, на связный список) без модификации остального кода.

Дерево (список смежности)

Дерево реализовано в виде класса Tree, использующего два словаря:

- `neighbors` - словарь, где ключ - номер узла, значение - список соседних узлов
- `letters` - словарь, где ключ - номер узла, значение - количество писем

Словари выбраны для хранения данных об узлах, поскольку они обеспечивают доступ по ключу (номеру узла) за время $O(1)$ в среднем. Это важно при обработке до 1000 гнезд, так

как позволяет быстро получать информацию о любом узле по его номеру. Кроме того, словари удобны тем, что не требуют предварительного знания количества элементов и автоматически расширяются при добавлении новых данных.

Архитектура программы

Программа разделена на три модуля:

- `stack.py` - содержит класс `Stack` и функцию `evaluate()`. Отвечает за вычисление математических выражений с операциями минимума и максимума. Не зависит от других модулей.
- `tree.py` - содержит класс `Tree` с методом `min_apologies()`. Отвечает за построение дерева гнезд и вычисление минимального количества извинений. Не зависит от других модулей.
- `main.py` - содержит пользовательский интерфейс: главное меню, подменю задач, функции ввода-вывода, проверки корректности данных и обработку ошибок. Импортирует функции из `stack.py` и `tree.py`.

Такое разделение позволяет изменять логику вычислений независимо от интерфейса, упрощает тестирование и повторное использование кода.

Пользовательский интерфейс

Программа запускается в консоли и предоставляет циклическое текстовое меню с нумерованными пунктами.

Главное меню

```
=====
ГЛАВНОЕ МЕНЮ
=====
1. Задача 1 – вычисление выражения
2. Задача 2 – почтальон Тритон
3. Выход
```

При вводе несуществующего пункта выводится сообщение «Ошибка: неверный пункт», меню отображается снова. Пункт 3 завершает программу.

Подменю задачи 1

```
-----
Задача 1: Вычисление выражения
m(a,b) – минимум, M(a,b) – максимум
Пример: M(15,m(16,8))
Пустая строка – возврат в меню
-----
Введите выражение: 
```

Пользователь вводит выражение и получает результат. Предусмотрены проверки:

- недопустимые символы — выводится список нарушений
- несбалансированные скобки — сообщение об ошибке
- пустая строка — возврат в главное меню

Подменю задачи 2

```
-----
Задача 2: Почтальон Тритон
-----
1. Загрузить файл и посчитать
2. Показать пример файла
3. В главное меню
> 
```

Пункт 1 запрашивает имя файла (по умолчанию input.txt), вычисляет и выводит результат. Пункт 2 показывает пример входных данных и формат. Обработываются ошибки: файл не найден, пустой файл, M вне диапазона, несоответствие количества строк.

Тестирование

Блок 1. Задача 1 — базовые сценарии

Тест 1.1. Пример из задания

```
Задача 1: Вычисление выражения
m(a,b) – минимум, M(a,b) – максимум
Пример: M(15,m(16,8))
Пустая строка – возврат в меню
-----
Введите выражение: M(15,m(16,8))
Результат: 15
```


Тест 1.2. Простой минимум

```
Задача 1: Вычисление выражения
m(a,b) – минимум, M(a,b) – максимум
Пример: M(15,m(16,8))
Пустая строка – возврат в меню
-----
Введите выражение: m(10,5)
Результат: 5
```

Тест 1.3. Простой максимум

```
Задача 1: Вычисление выражения
m(a,b) – минимум, M(a,b) – максимум
Пример: M(15,m(16,8))
Пустая строка – возврат в меню
-----
Введите выражение: M(10,5)
Результат: 10
```

Тест 1.4. Две вложенные операции

```
Задача 1: Вычисление выражения
m(a,b) – минимум, M(a,b) – максимум
Пример: M(15,m(16,8))
Пустая строка – возврат в меню
-----
Введите выражение: M(m(1,2),m(3,4))
Результат: 3
```

- Вход: $M(m(1,2),m(3,4))$
- Ожидаемый результат: 3
- Пройден.

Блок 2. Задача 1 — граничные случаи

Тест 2.1. Одинаковые аргументы

```
Задача 1: Вычисление выражения
m(a,b) – минимум, M(a,b) – максимум
Пример: M(15,m(16,8))
Пустая строка – возврат в меню
-----
Введите выражение: m(7,7)
Результат: 7
```

- Вход: $m(7,7)$
- Ожидаемый результат: 7
- Пройден.

Блок 3. Задача 1 — обработка некорректных данных

Тест 3.1. Недопустимый символ

```
Задача 1: Вычисление выражения
m(a,b) — минимум, M(a,b) — максимум
Пример: M(15,m(16,8))
Пустая строка — возврат в меню
-----
Введите выражение: K(15,8)
Ошибка: недопустимые символы ['K']
```

- Вход: K(15,8)
- Ожидаемый результат: ошибка «недопустимые символы ['K']»
- Пройден.

Тест 3.2. Несбалансированные скобки

```
Задача 1: Вычисление выражения
m(a,b) — минимум, M(a,b) — максимум
Пример: M(15,m(16,8))
Пустая строка — возврат в меню
-----
Введите выражение: M(15,m(16,8)
Ошибка: скобки не сбалансированы
```

- Вход: M(15,m(16,8))
- Ожидаемый результат: ошибка «скобки не сбалансированы»
- Пройден.

Тест 3.3. Посторонние символы

```
Задача 1: Вычисление выражения
m(a,b) — минимум, M(a,b) — максимум
Пример: M(15,m(16,8))
Пустая строка — возврат в меню
-----
Введите выражение: M(15,m(16,8)) + 5
Ошибка: недопустимые символы [' ', '+', ' ']
```

- Вход: M(15, m(16,8)) + 5
- Ожидаемый результат: ошибка «недопустимые символы»
- Пройден.

Тест 3.4. Пустая строка

```
Задача 1: Вычисление выражения
m(a,b) – минимум, M(a,b) – максимум
Пример: M(15,m(16,8))
Пустая строка – возврат в меню
-----
Введите выражение:
=====
ГЛАВНОЕ МЕНЮ
=====
1. Задача 1 – вычисление выражения
2. Задача 2 – почтальон Тритон
3. Выход
> 
```

- Вход: пустая строка
- Ожидаемый результат: возврат в главное меню
- Пройден.

Блок 4. Задача 2 — базовые сценарии

Тест 4.1. Пример из задания

```
≡ input.txt
1  5
2  3 2 2 5 4
3  1 1 1
4  1 1 4
5  2 2 3 1
6  1 3 1

ПРОБЛЕМЫ  ВЫХОДНЫЕ ДАННЫЕ  КОНСОЛЬ ОТ

-----
Задача 2: Почтальон Тритон
-----
1. Загрузить файл и посчитать
2. Показать пример файла
3. В главное меню
> 1
Имя файла: input.txt
Минимум извинений: 2
```

- Файл input.txt (5 гнезд, все с письмами)
- Результат: 2

Тест 4.2. Все без писем

```
test_empty.txt
1      3
2      2 0 2 3
3      1 0 1
4      1 0 1

ПРОБЛЕМЫ    ВЫХОДНЫЕ ДАННЫЕ    КОНСОЛЬ ОТЛАДКИ

-----
Задача 2: Почтальон Тритон
-----
1. Загрузить файл и посчитать
2. Показать пример файла
3. В главное меню
> 1
Имя файла: test_empty.txt
Минимум извинений: 0
```

- Файл test_empty.txt (3 гнезда, 0 писем)
- Результат: 0

Тест 4.3. Одно гнездо

```
test_one.txt
1      1
2      0 5

ПРОБЛЕМЫ    ВЫХОДНЫЕ ДАННЫЕ    КОНСОЛЬ ОТЛАДКИ

-----
Задача 2: Почтальон Тритон
-----
1. Загрузить файл и посчитать
2. Показать пример файла
3. В главное меню
> 1
Имя файла: test_one.txt
Минимум извинений: 0
```

- Файл test_one.txt (1 гнездо, 5 писем)
- Результат: 0

Блок 5. Задача 2 — граничные случаи

Тест 5.1. Корень без писем, дети с письмами

```
test_room_empty.txt
1 3
2 2 0 2 3
3 1 1 1
4 1 2 1

ПРОБЛЕМЫ    ВЫХОДНЫЕ ДАННЫЕ    КОНСОЛЬ ОТЛАДКИ

2. Задача 2 — почтальон Тритон
3. Выход
> 2

-----
Задача 2: Почтальон Тритон
-----
1. Загрузить файл и посчитать
2. Показать пример файла
3. В главное меню
> 1
Имя файла: test_room_empty.txt
Минимум извинений: 2
```

- Файл test_root_empty.txt: узел 1 (0 писем), узлы 2 (1 письмо), 3 (2 письма)
- Результат: 2

Тест 5.2. Цепочка, письма в конце

```
≡ test_chain.txt
1  4
2  1 0 2
3  2 0 1 3
4  2 0 2 4
5  1 5 3

ПРОБЛЕМЫ  ВЫХОДНЫЕ ДАННЫЕ  КОНСОЛЬ ОТ

2. Задача 2 – почтальон Тритон
3. Выход
> 2
-----
Задача 2: Почтальон Тритон
-----
1. Загрузить файл и посчитать
2. Показать пример файла
3. В главное меню
> 1
Имя файла: test_chain.txt
Минимум извинений: 3
```

- Файл test_chain.txt: 1-2-3-4, письма только в 4
- Результат: 0

Тест 5.3. Три ветки от корня

```
test_three_branches.txt
1  4
2  3 1 2 3 4
3  1 1 1
4  1 1 1
5  1 1 1

ПРОБЛЕМЫ    ВЫХОДНЫЕ ДАННЫЕ    КОНСОЛЬ ОТ

2. Задача 2 — почтальон Тритон
3. Выход
> 2

-----
Задача 2: Почтальон Тритон
-----
1. Загрузить файл и посчитать
2. Показать пример файла
3. В главное меню
> 1
Имя файла: test_three_branches.txt
Минимум извинений: 2
-----
```

- Файл test_three_branches.txt: 1 связан с 2, 3, 4, у всех по 1 письму
- Результат: 2

Блок 6. Задача 2 — ошибки файлов

Тест 6.1. Файл не найден

```
-----
Задача 2: Почтальон Тритон
-----
1. Загрузить файл и посчитать
2. Показать пример файла
3. В главное меню
> 1
Имя файла: nofile.txt
Ошибка: файл 'nofile.txt' не найден
-----
```

- Вход: несуществующий файл
- Результат: ошибка «файл не найден»

Тест 6.2. Пустой файл

```
-----  
Задача 2: Почтальон Тритон  
-----
```

1. Загрузить файл и посчитать
2. Показать пример файла
3. В главное меню

```
> 1
```

```
Имя файла: empty.txt
```

```
Ошибка: файл пуст
```

- Вход: файл без строк
- Результат: ошибка «файл пуст»

Тест 6.3. $M = 0$

```
≡ empty.txt
```

```
1  0
```

```
ПРОБЛЕМЫ  ВЫХОДНЫЕ ДАННЫЕ  КОНСОЛЬ ОТ
```

```
3. Выход
```

```
> 2
```

```
-----  
Задача 2: Почтальон Тритон  
-----
```

1. Загрузить файл и посчитать
2. Показать пример файла
3. В главное меню

```
> 1
```

```
Имя файла: empty.txt
```

```
Ошибка: M должно быть от 1 до 999
```

- Файл: первая строка 0
- Результат: ошибка «M должно быть от 1 до 999»

Тест 6.4. Несоответствие строк

```
≡ empty.txt
1 5
2 1

ПРОБЛЕМЫ    ВЫХОДНЫЕ ДАННЫЕ    КОНСОЛЬ ОТ

> 2
-----
Задача 2: Почтальон Тритон
-----
1. Загрузить файл и посчитать
2. Показать пример файла
3. В главное меню
> 1
Имя файла: empty.txt
Ошибка: строк 2, нужно 6
```

- Файл: $M = 5$, строк 2
- Результат: ошибка «строк 2, нужно 6»

Блок 7. Интерфейс

Тест 7.1. Неверный пункт меню

```
=====
ГЛАВНОЕ МЕНЮ
=====
1. Задача 1 – вычисление выражения
2. Задача 2 – почтальон Тритон
3. Выход
> 5
Ошибка: неверный пункт
=====
ГЛАВНОЕ МЕНЮ
=====
1. Задача 1 – вычисление выражения
2. Задача 2 – почтальон Тритон
3. Выход
> 
```

- Вход: 5 в главном меню
- Результат: ошибка, меню снова

Тест 7.2. Возврат из подменю

<pre>===== ГЛАВНОЕ МЕНЮ ===== 1. Задача 1 – вычисление выражения 2. Задача 2 – почтальон Тритон 3. Выход > 1 ----- Задача 1: Вычисление выражения m(a,b) – минимум, M(a,b) – максимум Пример: M(15,m(16,8)) Пустая строка – возврат в меню ----- Введите выражение: ===== ГЛАВНОЕ МЕНЮ ===== 1. Задача 1 – вычисление выражения 2. Задача 2 – почтальон Тритон 3. Выход > </pre>	<pre>===== ГЛАВНОЕ МЕНЮ ===== 1. Задача 1 – вычисление выражения 2. Задача 2 – почтальон Тритон 3. Выход > 2 ----- Задача 2: Почтальон Тритон ----- 1. Загрузить файл и посчитать 2. Показать пример файла 3. В главное меню > 3 ===== ГЛАВНОЕ МЕНЮ ===== 1. Задача 1 – вычисление выражения 2. Задача 2 – почтальон Тритон 3. Выход > </pre>
---	--

- Вход: пустая строка в задаче 1, пункт 3 в задаче 2
- Результат: возврат в главное меню

Тест 7.3. Выход из программы

```
=====
ГЛАВНОЕ МЕНЮ
=====
1. Задача 1 – вычисление выражения
2. Задача 2 – почтальон Тритон
3. Выход
> 3
Выход
PS C:\Users\Antipin\Desktop\GAME\bot>
```

- Вход: 3 в главном меню
- Результат: завершение программы

Анализ результатов

Программа полностью соответствует поставленным задачам. Алгоритм вычисления выражения работает за линейное время $O(n)$, алгоритм поиска извинений — за $O(M)$. Все ошибки ввода перехватываются, программа не завершается аварийно. Модульная архитектура обеспечивает четкое разделение ответственности. Код соответствует PEP 8. Собственные классы `Stack` и `Tree` удовлетворяют требованию работы с динамическими структурами данных.

Код программы

Ссылка на гит - [EmptyQwazar/Individualnoe_zadanie_2_sem_pyhon](https://github.com/EmptyQwazar/Individualnoe_zadanie_2_sem_pyhon): В данном репозитории хранятся работы по двум индивидуальным заданиям по предмету 'Язык программирования Python' ПГНИУ. Выполнил задания студент ПГНИУ, 1 курс ИКНТ, группа ИТ-7,8, Антипин Дмитрий Константинович.

stack.py

```
class Stack:

    def __init__(self):

        self._items = []

    def push(self, item):

        self._items.append(item)

    def pop(self):

        if self.is_empty():

            raise IndexError("Стек пуст")

        return self._items.pop()

    def is_empty(self):

        return len(self._items) == 0
```

```
def evaluate(expr):

    stack = Stack()

    i = 0

    while i < len(expr):

        c = expr[i]

        if c in 'mM':

            stack.push(c)

            i += 1

        elif c.isdigit():

            num = ''

            while i < len(expr) and expr[i].isdigit():

                num += expr[i]

                i += 1

            stack.push(int(num))

        elif c == ')':

            b = stack.pop()
```

```

a = stack.pop()

op = stack.pop()

stack.push(min(a, b) if op == 'm' else max(a, b))

i += 1

else:

    i += 1

return stack.pop()

```

tree.py

```

class Tree:

    def __init__(self):

        self.neighbors = {}

        self.letters = {}

    def add_node(self, node_id, letters_count, neighbors_list):

        self.letters[node_id] = letters_count

        self.neighbors[node_id] = neighbors_list

    def min_apologies(self):

```

```
visited = set()
```

```
apologies = 0
```

```
def dfs(v, parent):
```

```
    nonlocal apologies
```

```
    visited.add(v)
```

```
    children_with_mail = 0
```

```
    for to in self.neighbors[v]:
```

```
        if to != parent:
```

```
            if dfs(to, v):
```

```
                children_with_mail += 1
```

```
    if self.letters[v] > 0:
```

```
        if children_with_mail > 1:
```

```
            apologies += children_with_mail - 1
```

```
        return True
```

```
    else:
```

```
        if children_with_mail > 0:
```

```
        apologies += children_with_mail

    return True

    return False

dfs(1, -1)

return apologies
```

main.py

```
from stack import evaluate

from tree import Tree

def task1():

    print("-" * 40)

    print("Задача 1: Вычисление выражения")

    print("  m(a,b) — минимум, M(a,b) — максимум")

    print("  Пример: M(15, m(16,8))")

    print("  Пустая строка — возврат в меню")

    print("-" * 40)
```

```
while True:

    expr = input("Введите выражение: ").strip()

    if not expr:

        break

    allowed = set('mM(),0123456789')

    bad = [c for c in expr if c not in allowed]

    if bad:

        print(f"Ошибка: недопустимые символы {bad}")

        continue

    if expr.count('(') != expr.count(')'):

        print("Ошибка: скобки не сбалансированы")

        continue

    try:

        print("Результат:", evaluate(expr))

    except Exception as e:

        print(f"Ошибка: {e}")
```



```
def task2():

    while True:

        print("-" * 40)

        print("Задача 2: Почтальон Тритон")

        print("-" * 40)

        print("1. Загрузить файл и посчитать")

        print("2. Показать пример файла")

        print("3. В главное меню")

        choice = input("> ").strip()

        if choice == '1':

            filename = input("Имя файла: ").strip() or 'input.txt'

            try:

                with open(filename, encoding='utf-8') as f:

                    lines = f.readlines()

                    if not lines:

                        print("Ошибка: файл пуст")
```

```

        continue

M = int(lines[0].strip())

if M <= 0 or M >= 1000:

    print("Ошибка: М должно быть от 1 до 999")

    continue

if len(lines) < M + 1:

    print(f"Ошибка: строка {len(lines)}, нужно {M + 1}")

    continue

tree = Tree()

for i in range(1, M + 1):

    data = list(map(int, lines[i].split()))

    tree.add_node(i, data[1], data[2:])

print("Минимум извинений:", tree.min_apologies())

except FileNotFoundError:

    print(f"Ошибка: файл '{filename}' не найден")

except ValueError as e:

```

```
        print(f"Ошибка: {e}")

elif choice == '2':

    print("Пример input.txt:")

    print("5")

    print("3 2 2 5 4")

    print("1 1 1")

    print("1 1 4")

    print("2 2 3 1")

    print("1 3 1")

    print("Формат:")

    print("  Строка 1: M — количество гнезд")

    print("  Строки 2..M+1: ni li [соседи...]")

elif choice == '3':

    break

else:

    print("Ошибка: неверный пункт")
```

```
def main():

    while True:

        print("=" * 40)

        print("ГЛАВНОЕ МЕНЮ")

        print("=" * 40)

        print("1. Задача 1 — вычисление выражения")

        print("2. Задача 2 — почтальон Тритон")

        print("3. Выход")

        choice = input("> ").strip()

        if choice == '1':

            task1()

        elif choice == '2':

            task2()

        elif choice == '3':
```

```
        print("Выход")

        break

    else:

        print("Ошибка: неверный пункт")

if __name__ == "__main__":

    main()
```